

Optional

- ❗ "Optional is a container type for return values that forces callers to handle the absent case — it replaces null with an explicit API contract."

Without Optional — NPE waiting to happen

```
1 // findUser returns null if not found
2 User u = db.findUser(id);
3 u.getName(); // NullPointerException!
4
5 // You must remember to check every time
6 if (u != null) {
7     u.getName();
8 }
```

With Optional — null is part of the type

```
1 // return type SAYS it might be absent
2 Optional<User> u = db.findUser(id);
3
4 // compiler forces you to handle absence
5 u.ifPresent(user ->
6     System.out.println(user.getName())
7 );
```

- ❗ Optional = boîte qui force à gérer l'absence.

3 ways to create an Optional

```
1 // 1. of() - value is GUARANTEED non-null
2 //   throws NullPointerException if you pass null!
3 Optional<String> a = Optional.of("hello");
4 Optional<String> x = Optional.of(null);    // NPE!
5
6 // 2. ofNullable() - value MAY be null
7 //   wraps it if present, returns empty if null
8 Optional<String> b = Optional.ofNullable(maybeNull);
9
10 // 3. empty() - explicitly absent
11 Optional<String> c = Optional.empty();
```

- ❗ use `of` when you KNOW it's not null (e.g. a constant). Use `ofNullable` when wrapping values from legacy code or external APIs.

Getting the value out — safe vs unsafe

```
1 Optional<String> opt = Optional.of("Java");
2 Optional<String> empty = Optional.empty();
3
4 // ✓ get() - UNSAFE, throws if empty
5 opt.get();    // "Java"
6 empty.get();  // NoSuchElementException!
```

```

7
8 // ✓ orElse() – default value (always evaluated)
9 empty.orElse("default");    // "default"
10
11 // ✓ orElseGet() – lazy supplier (only called if empty)
12 empty.orElseGet(() -> compute()); // cheaper!
13
14 // ✓ orElseThrow() – throw custom exception
15 empty.orElseThrow(() ->
16     new NotFoundException("not found"));
17
18 // ✓ ifPresent() – run action only if value exists
19 opt.ifPresent(v -> System.out.println(v));
20
21 // ✓ ifPresentOrElse() – Java 9+
22 opt.ifPresentOrElse(
23     v -> System.out.println(v),
24     () -> System.out.println("absent")
25 );

```

- ❗ Never use `get()` without first checking `isPresent()` — defeats the purpose of `Optional` entirely.

Transforming values — the functional core

```

1 // map() – transform the value if present
2 //     wraps result back in Optional
3 Optional<String> name = Optional.of("java");
4
5 Optional<String> upper = name
6     .map(String::toUpperCase);    // Optional["JAVA"]
7
8 Optional<Integer> len = name
9     .map(String::length);        // Optional[4]
10
11 Optional<String> empty = Optional.<String>empty();
12 empty.map(String::toUpperCase);  // Optional.empty() – no NPE
13
14
15 // flatMap() – use when the mapper itself
16 //     returns an Optional
17 //     avoids Optional<Optional<T>>
18
19 class User {
20     Optional<Address> getAddress() { ... }
21 }
22
23 // map gives Optional<Optional<Address>> – wrong!
24 user.map(User::getAddress);
25
26 // flatMap gives Optional<Address> – correct!
27 user.flatMap(User::getAddress)
28     .map(Address::getCity)
29     .orElse("unknown");

```

- ❗ Rule: if your mapper returns `Optional<X>` → use `flatMap`. If it returns `X` → use `map`.

Filtering and chaining Optionals

```

1 // filter() - keep value only if predicate passes
2 //           returns empty if predicate fails
3 Optional<Integer> age = Optional.of(17);
4
5 age.filter(a -> a >= 18); // Optional.empty()
6 age.filter(a -> a >= 10); // Optional[17]
7
8 -----
9 // or() - Java 9+ fallback to another Optional
10 //       like orElseGet() but returns Optional
11 Optional<String> primary = Optional.empty();
12 Optional<String> secondary = Optional.of("backup");
13
14 primary.or(() -> secondary); // Optional["backup"]
15
16 -----
17 // stream() - Java 9+ bridge to Stream API
18 //           0 or 1 element stream
19 List<Optional<String>> list = ...;
20
21 list.stream()
22     .flatMap(Optional::stream) // flatten all presents
23     .collect(toList());

```

Real-world chaining — the power of Optional

```

1 // Goal: get city of user's address, or "unknown"
2
3 // Old Java 8 style (nested null checks)
4 String city = "unknown";
5 if (user != null) {
6     Address addr = user.getAddress();
7     if (addr != null) {
8         String c = addr.getCity();
9         if (c != null) city = c;
10    }
11 }
12
13 -----
14 // With Optional - clean pipeline
15 String city = Optional.ofNullable(user)
16     .flatMap(u -> u.getAddress()) // Optional<Address>
17     .map(addr -> addr.getCity()) // Optional<String>
18     .filter(c -> !c.isBlank()) // guard
19     .orElse("unknown"); // default
20
21 -----
22 // With Java 21 + record pattern (bonus)
23 record Address(String street, String city) {}
24 record User(String name, Address address) {}
25
26 String city = findUser(id)
27     .map(User::address)
28     .map(Address::city)
29     .orElseThrow(UserNotFoundException::new);

```

What NOT to do

Do NOT use Optional as a field in a class — it is not Serializable

```

1 class User {
2     Optional<String> email; // wrong! not serializable
3 }

```

```

4 class User {
5     String email; // nullable field is fine
6     Optional<String> getEmail() { return Optional.ofNullable(email); }
7 }

```

Do NOT use Optional as a method parameter

```

1 void send(Optional<String> email) { ... } // wrong
2 void send(String email) { ... }           // nullable param is cleaner

```

Do NOT use isPresent() + get() — use map/ifPresent instead

```

1 if (opt.isPresent()) { return opt.get(); } // verbose
2 return opt.orElse(defaultValue);          // clean
3
4 // instead of this
5 if (opt.isPresent()) { send(opt.get()); }
6 // use this
7 opt.ifPresent(v -> send(v));
8
9 // instead of this
10 if (opt.isPresent()) { return opt.get().toUpperCase(); }
11 // use this
12 return opt.map(String::toUpperCase).orElse("default");

```

Optional is designed for return types only — to signal "this method might not find a result".

CREATE	UNWRAP
of(x) — non-null guaranteed	orElse(v) — default value
ofNullable(x) — may be null	orElseGet(()->v) — lazy default
empty() — explicitly absent	orElseThrow(()->ex) — throw
TRANSFORM	ifPresent(c) — side effect
map(f) — f returns T	get() — avoid!
flatMap(f) — f returns Optional<T>	JAVA 9+
filter(p) — keep if predicate true	or(()->opt) — fallback Optional
	stream() — bridge to Stream
	ifPresentOrElse(c, r)